

SQL - Intermediaire

Sous-requetes, dates et rapports utiles.



Sous-requetes



Dates



Rapports



FORMATION SQL - INTERMEDIAIRE

SQL - Intermediaire

Sous-requetes, dates, HAVING et rapports
utiles, apres les bases.

DanielCraft - Livre debutant - Pack Data/SQL

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [Chapitre 1 - Rappel des bases](#)
 - [Petite histoire](#)
 - [A toi](#)
2. [Chapitre 2 - Sous-requetes](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [A toi](#)
3. [Chapitre 3 - IN et EXISTS](#)
 - [A toi](#)
4. [Chapitre 4 - CASE WHEN](#)
 - [A toi](#)
5. [Chapitre 5 - Fonctions texte](#)
 - [A toi](#)
6. [Chapitre 6 - Dates et periodes](#)
 - [Petite histoire](#)
 - [A toi](#)
7. [Chapitre 7 - HAVING](#)
 - [A toi](#)
8. [Chapitre 8 - UNION](#)
 - [A toi](#)
9. [Chapitre 9 - Vues \(idee\)](#)
 - [A toi](#)
10. [Chapitre 10 - Index \(idee\)](#)
 - [A toi](#)
11. [Chapitre 11 - Transactions \(idee\)](#)
 - [A toi](#)
12. [Chapitre 12 - Fonctions fenetre \(idee\)](#)
 - [A toi](#)
13. [Chapitre 13 - Mini-projet : rapport du mois](#)
 - [A toi](#)
14. [Chapitre 14 - Retenir l'essentiel](#)
15. [Chapitre 15 - Atelier : sous-requetes](#)
16. [Chapitre 16 - Atelier : dates](#)
17. [Chapitre 17 - Atelier : rapport](#)
18. [Chapitre 18 - Erreurs classiques](#)
19. [Chapitre 19 - Bonnes pratiques](#)
20. [Quiz final](#)

- [Question 1](#)
- [Question 2](#)
- [Question 3](#)
- [Question 4](#)
- [Question 5](#)
- [Question 6](#)
- [Question 7](#)
- [Question 8](#)
- [Question 9](#)
- [Question 10](#)
- [Question 11](#)
- [Question 12](#)
- [Corrige](#)

21. [Bravo](#)

- [Suite](#)

Chapitre 1 - Rappel des bases



Rappel : SELECT WHERE JOIN, puis on monte d'un cran.

Tu as déjà **SELECT**, **WHERE**, **JOIN**, **GROUP BY**, **NULL**. Ce livre **intermediaire** ajoute des pieces pour des questions plus riches : sous-requetes, dates, **HAVING**, unions, et un peu d'outils (vues, index, transactions) en mode idee. Chez DanielCraft, on ne remplace pas le livre bases : on s'appuie dessus. Lea relit un JOIN avant chaque sous-requete. Max oublie parfois **WHERE** vs **HAVING** : on y revient. Sam dit : "plus de puissance, plus de verification."

★ A RETENIR

Intermediaire = bases solides + outils pour des questions metier plus fines.

Petite histoire

Lea devait lister les clients qui ont commande plus de 3 fois. En bases seules, elle galerait. Avec sous-requete ou **HAVING**, elle livre. Max ajoute un filtre "ce mois". Sam verifie trois lignes a la main.

A toi

Ecris trois questions que tu ne savais pas poser proprement apres le livre bases.

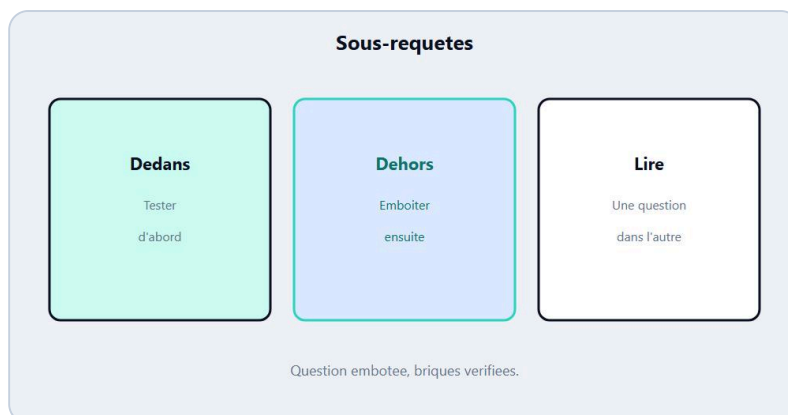
⚠ ATTENTION

Sans bases, ce livre sera flou. Relis JOIN et GROUP BY si besoin.

♦ ASTUCE

Garde un schema `clients` / `commandes` sous les yeux.

Chapitre 2 - Sous-requetes



Sous-requete : une question dans la question.

Une **sous-requete** est une requete rangee dans une autre. Souvent dans **WHERE**, parfois dans **FROM** ou **SELECT**. Chez DanielCraft : ecrire d'abord la question interieure, tester, puis l'emboiter. Lea filtre les **client_id** actifs, puis joint. Max colle tout d'un coup et se perd.

```
SELECT prenom
FROM clients
WHERE id IN (
  SELECT client_id
  FROM commandes
  WHERE montant > 100
);
```

★ A RETENIR

Sous-requete = question dans la question. Teste le dedans d'abord.



Exemple : Lea filtre d'abord, puis croise.

Petite histoire

Sam demande "clients avec au moins une commande > 100". Lea ecrit le **SELECT** interieur, voit 12 id, puis **IN**. Max veut tout en un **JOIN** : parfois equivalent, parfois plus clair en sous-requete.

Erreur classique

Sous-requete qui renvoie plusieurs colonnes la ou `IN` attend une. Ou correlee lourde sans besoin.

A toi

Ecris une sous-requete : ids des commandes d'un client donne, puis les details.

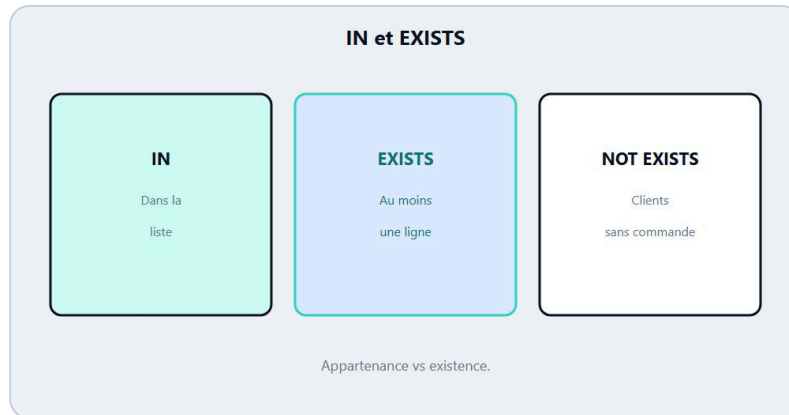
⚠ ATTENTION

Une sous-requete fausse empoisonne toute la requete mere.

♦ ASTUCE

Alias clairs : `c` clients, `cmd` commandes.

Chapitre 3 - IN et EXISTS



IN et EXISTS : tester l'appartenance / l'existence.

IN teste si une valeur est dans une liste / resultat. **EXISTS** teste s'il existe au moins une ligne. Chez DanielCraft, on lit l'intention : appartenance vs existence. Lea prefere **EXISTS** quand elle verifie "a deja commande". Max utilise **IN** pour une petite liste d'ids.

```
SELECT prenom FROM clients c
WHERE EXISTS (
  SELECT 1 FROM commandes cmd
  WHERE cmd.client_id = c.id
);
```

★ A RETENIR

IN = dans la liste. EXISTS = il existe au moins une ligne.

A toi

Reecris une question "clients sans commande" avec **NOT EXISTS**.

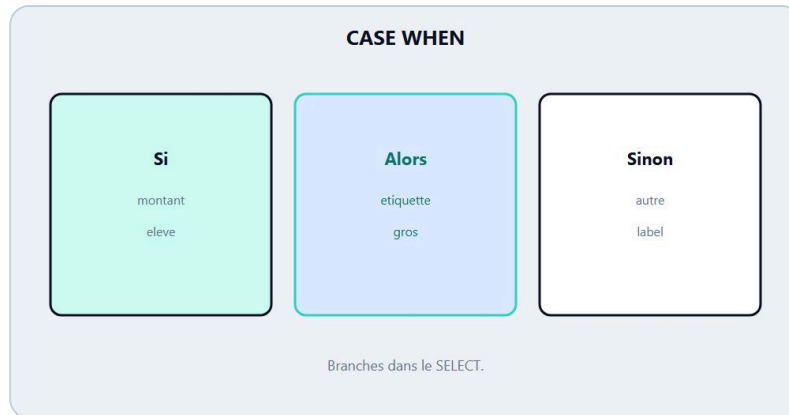
⚠ ATTENTION

NOT IN avec des NULL dans la sous-liste peut surprendre. Prefere **NOT EXISTS** si tu hesites.

♦ ASTUCE

SELECT 1 dans EXISTS suffit : on teste l'existence, pas les colonnes.

Chapitre 4 - CASE WHEN



CASE WHEN : des branches dans le SELECT.

CASE WHEN cree des branches dans le resultat : labels, paniers, statuts. Chez DanielCraft, c'est le "si" du SELECT. Lea marque **petit** / **gros** selon le montant. Max evite dix requetes separees.

```
SELECT id, montant,
CASE
  WHEN montant >= 100 THEN 'gros'
  WHEN montant >= 50 THEN 'moyen'
  ELSE 'petit'
END AS panier
FROM commandes;
```

★ A RETENIR

CASE WHEN = etiquettes calculees dans le SELECT.

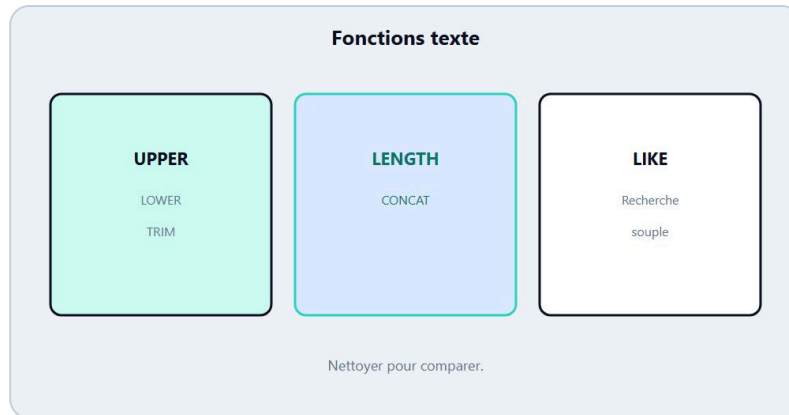
A toi

Etiquette les clients : 'actif' s'ils ont une commande (idee : sous-requete ou jointure + CASE).

♦ ASTUCE

Ordre des WHEN : du plus precis au plus large.

Chapitre 5 - Fonctions texte



Texte : CONCAT, LENGTH, UPPER, LIKE avance.

UPPER, LOWER, LENGTH, TRIM, CONCAT (ou || selon moteur), LIKE avec %. Chez DanielCraft : nettoyer et comparer sans magie. Lea uniformise les villes en majuscules pour croiser. Max cherche LIKE '%Paris%'.

Les noms exacts dependent du moteur (SQLite, Postgres, MySQL). L'idée reste : transformer le texte pour filtrer ou afficher.

★ A RETENIR

Fonctions texte = nettoyer, mesurer, concatener, chercher.

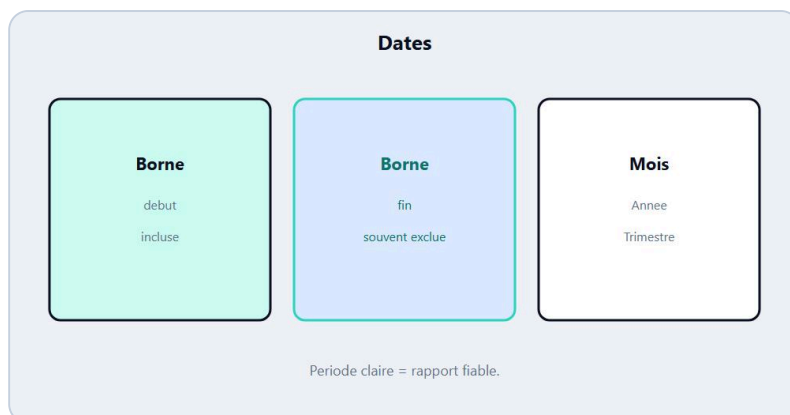
A toi

Ecris un filtre : prenom qui commencent par 'A' (LIKE).

⚠ ATTENTION

LIKE sensible a la casse selon le moteur / collation.

Chapitre 6 - Dates et periodes



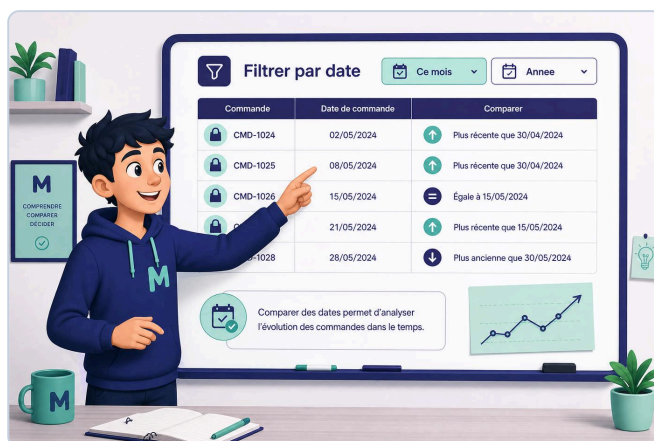
Dates : filtrer un mois, une annee, un intervalle.

Filtrer "ce mois", "cette annee", "entre deux dates". Chez DanielCraft, on stocke des dates propres et on filtre avec des bornes. Lea coupe `date_cmd >= '2026-07-01' AND date_cmd < '2026-08-01'`. Max utilise des fonctions (`DATE_TRUNC`, `strftime` ...) selon le moteur - l'idée compte plus que le dialecte.

```
SELECT * FROM commandes
WHERE date_cmd >= '2026-07-01'
      AND date_cmd < '2026-08-01';
```

★ A RETENIR

Periode claire = borne basse incluse, borne haute souvent exclue.



Exemple : Max coupe les commandes du mois.

Petite histoire

Sam livre un CA "juillet". Sans bornes nettes, aout fuyait dedans. Lea impose l'intervalle demi-ouvert. Max arrete les `LIKE '2026-07%'` sur des timestamps.

A toi

Ecris le filtre pour le trimestre invente Q1 2026.

⚠ ATTENTION

Comparer date et texte mal formate = tris foireux.

♦ ASTUCE

Documente le fuseau / la convention "jour calendaire" du projet.

Chapitre 7 - HAVING



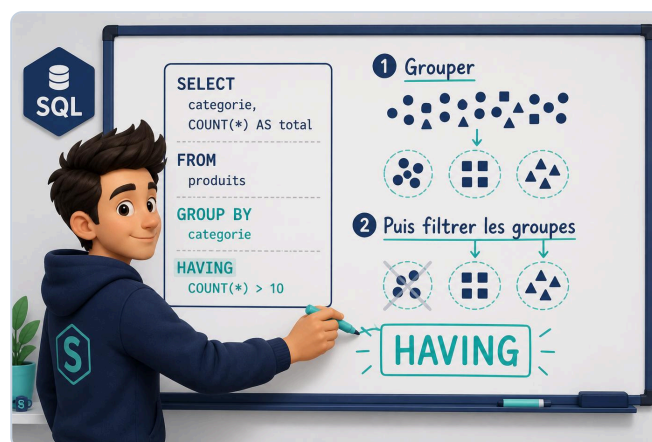
HAVING filtre les groupes apres GROUP BY.

WHERE filtre les lignes **avant** regroupement. HAVING filtre les **groupes** apres GROUP BY. Chez DanielCraft, Lea dit : "d'abord les lignes, ensuite les paquets". Max voulait WHERE COUNT() > 3 : *refuse*. HAVING COUNT() > 3 : *oui*.

```
SELECT client_id, COUNT(*) AS nb
FROM commandes
GROUP BY client_id
HAVING COUNT(*) >= 3;
```

★ A RETENIR

WHERE = lignes. HAVING = groupes.



Exemple : Sam garde les groupes au-dessus d'un seuil.

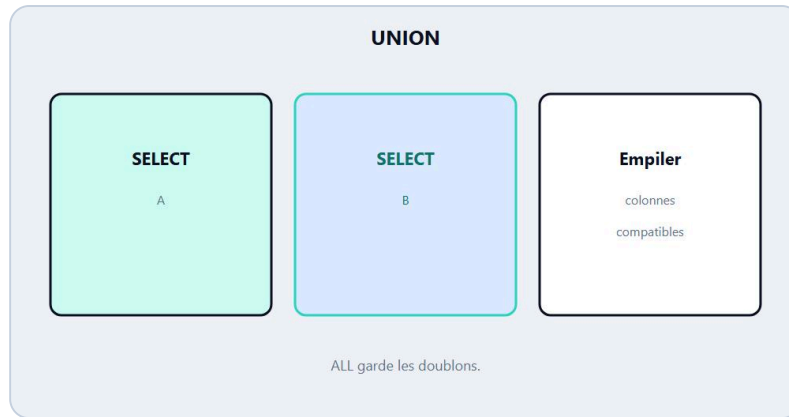
A toi

Villes avec un total de montants > 500 (JOIN + GROUP BY + HAVING).

⚠ ATTENTION

Mettre un agrégat dans WHERE est une erreur classique.

Chapitre 8 - UNION



UNION : empiler des resultats compatibles.

UNION empile des resultats de meme forme (memes colonnes compatibles). **UNION ALL** garde les doublons. Chez DanielCraft : utile pour fusionner archives + courant, ou deux sources proches. Lea aligne les alias. Max oublie le meme nombre de colonnes.

★ A RETENIR

UNION empile des SELECT compatibles. ALL conserve les doublons.

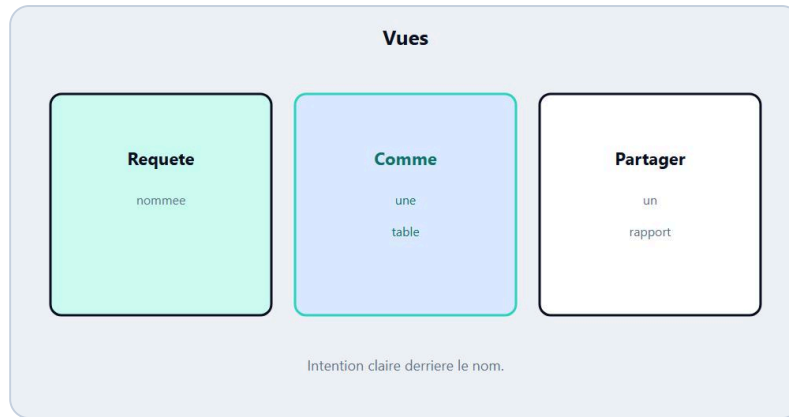
A toi

Empile les ids clients de Paris et de Lyon (deux SELECT).

♦ ASTUCE

Meme ordre et types de colonnes sur chaque SELECT.

Chapitre 9 - Vues (idée)



Vue : une requete enregistree, reutilisable.

Une **vue** enregistre une requete sous un nom. Tu la requetes comme une table. Chez DanielCraft : pour partager un rapport stable, pas pour cacher de la magie. Lea cree `vue_ca_client`. Max la SELECT ensuite.

★ A RETENIR

Vue = requete nommee, reutilisable.

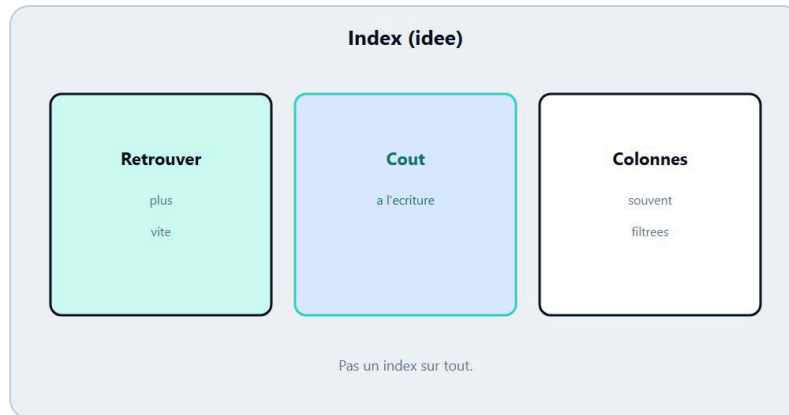
A toi

Decris en francais une vue utile pour ton mini schema.

⚠ ATTENTION

Une vue complexe peut masquer un cout : comprends la requete derriere.

Chapitre 10 - Index (idée)



Index : accélérer certaines recherches (idée).

Un **index** aide la base à retrouver des lignes plus vite (comme un index de livre). Chez DanielCraft : idée seulement. Trop d'index ralentit les écritures. Lea indexe `commandes.client_id` si les jointures sont fréquentes. Max n'indexe pas "au feeling" chaque colonne.

★ A RETENIR

Index = accélérer certaines lectures. Pas une obligation partout.

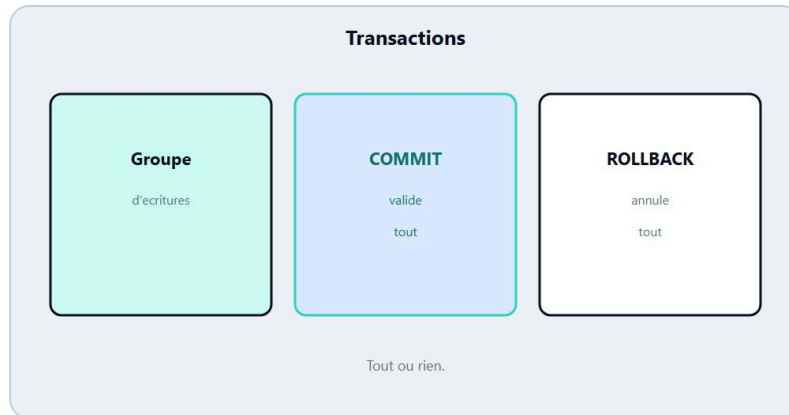
A toi

Quelle colonne filtres-tu le plus souvent ?

⚠ ATTENTION

L'index n'est pas un remède à une mauvaise requête.

Chapitre 11 - Transactions (idee)



Transaction : tout reussit ou tout annule (idee).

Une **transaction** groupe des changements : tout valide (**COMMIT**) ou tout annule (**ROLLBACK**). Chez DanielCraft : idee pour transferer / reserver sans etat a moitie. Lea enveloppe debit + credit. Max apprend a ne pas laisser une session ouverte indefiniment.

★ A RETENIR

Transaction = tout ou rien pour un groupe d'ecritures.

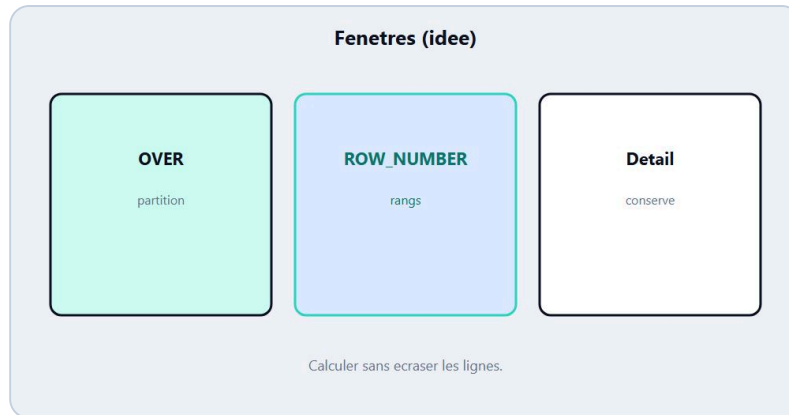
A toi

Cite un cas ou "a moitie ecrit" serait grave.

⚠ ATTENTION

Ce livre ne remplace pas le cours locking / isolation avance.

Chapitre 12 - Fonctions fenetre (idee)



Fonctions fenetre : rangs sans écraser les lignes (idee).

`ROW_NUMBER`, `RANK`, `SUM() OVER (...)` calculent sur un partitionnement **sans écraser** les lignes (contrairement à un `GROUP BY` qui agrège). Chez DanielCraft : idee pour top N par groupe. Lea numeroote les commandes par client. Sam dit "fenetre = regarder les voisins".

★ A RETENIR

Fenetre = calcul sur un groupe de lignes, en gardant le detail.

A toi

Decris : "rang de chaque commande par montant dans le mois".

♦ ASTUCE

Apprends `ROW_NUMBER` avant les fenetres complexes.

Chapitre 13 - Mini-projet : rapport du mois



Mini-projet : rapport clients / commandes avance.

Schema invente `clients` / `commandes`. Objectif : rapport juillet invente.

1. 1. CA total du mois (bornes de dates)
2. 2. Top 5 clients par montant (JOIN + GROUP BY + ORDER + LIMIT)
3. 3. Clients avec au moins 3 commandes (`HAVING`)
4. 4. Etiquette panier via `CASE`
5. 5. Bonus : clients sans commande du mois (`NOT EXISTS`)

Lea livre 1-2. Max bloque sur 3 puis relit `HAVING`. Sam verifie les bornes de dates.

★ A RETENIR

Rapport = questions + requetes + echantillon verifie.



Exemple : Lea, Max et Sam livrent le rapport du mois.

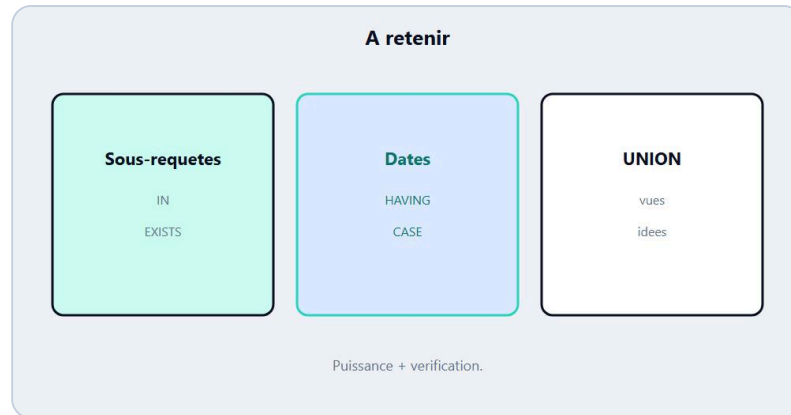
A toi

Une page "Rapport SQL inter - date" avec les 5 points.

 ATTENTION

Un beau total sans bornes de dates est suspect.

Chapitre 14 - Retenir l'essentiel



Sous-requetes, dates, HAVING : la carte inter.

Sous-requetes testees dedans-dehors. IN / EXISTS. CASE. Texte et dates avec bornes. HAVING apres GROUP BY. UNION aligne. Vues / index / transactions / fenetres en idee. Chez DanielCraft : puissance + verification.

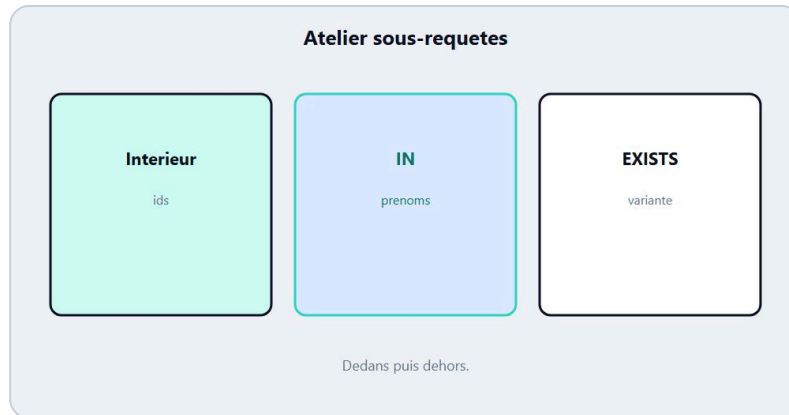
★ A RETENIR

WHERE lignes, HAVING groupes, sous-requete = question embotee.

♦ ASTUCE

Relis dates et HAVING avant tout CA mensuel.

Chapitre 15 - Atelier : sous-requetes



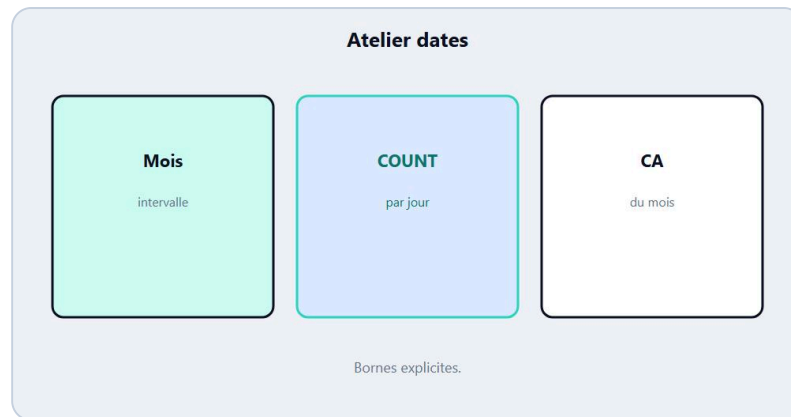
Atelier : ecrire une sous-requete claire.

1. Liste des `client_id` avec montant > 80
2. Prenoms correspondants via `IN`
3. Meme idee avec `EXISTS`
4. Clients sans commande (`NOT EXISTS`)

★ **A RETENIR**

Dedans d'abord, dehors ensuite.

Chapitre 16 - Atelier : dates



Atelier : filtrer et grouper par date.

1. 1. Commandes d'un mois invente (intervalle)
2. 2. COUNT par jour
3. 3. CA du mois
4. 4. Comparer deux mois (deux requetes ou UNION)

★ A RETENIR

Bornes de dates explicites.

Chapitre 17 - Atelier : rapport



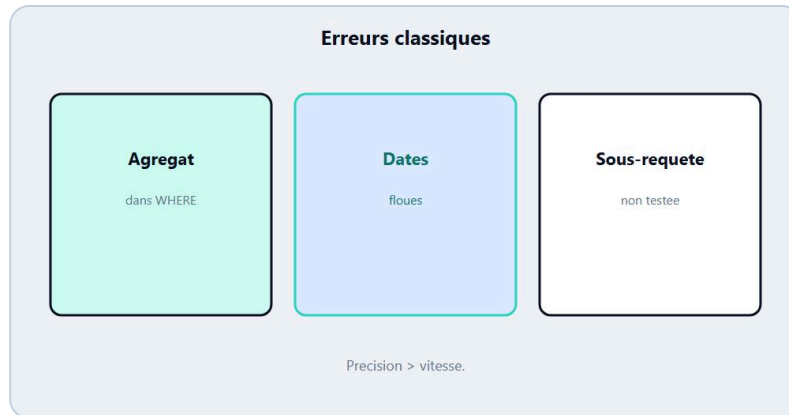
Atelier : un petit rapport metier.

Construis un mini rapport : titre, periode, 3 requetes, 3 chiffres verifies a la main sur un echantillon.

★ A RETENIR

Le rapport est un discours appuye sur des requetes.

Chapitre 18 - Erreurs classiques



Pieges : sous-requete correee lourde, HAVING vs WHERE.

Agregat dans WHERE. HAVING oublie. Sous-requete non testee. Bornes de dates floues. UNION desordonne. `NOT IN` + NULL. Index miracle.

★ A RETENIR

Les pieges inter sont des pieges de precision.

Chapitre 19 - Bonnes pratiques



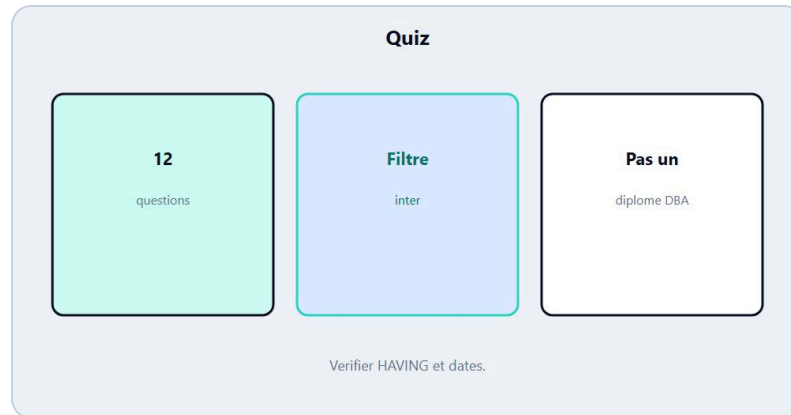
Pratiques : etapes, alias, echantillons.

1. 1. Ecrire la question en francais
2. 2. Tester les briques
3. 3. Alias stables
4. 4. Bornes de dates documentees
5. 5. Echantillon manuel
6. 6. Préférer EXISTS si NULL te stress
7. 7. Commenter la requete metier
8. 8. Limiter avant d'agreger en exploratoire
9. 9. Une vue = une intention claire
10. 10. Mesurer avant d'indexer

★ A RETENIR

Briques testees > monstre opaque.

Quiz final



Quiz SQL intermediaire.

Question 1

Une sous-requete est :

- A) Un index
- B) Une requete dans une requete
- C) Un DROP

Question 2

HAVING filtre :

- A) Les lignes avant GROUP BY
- B) Les groupes apres GROUP BY
- C) Les colonnes du SELECT *

Question 3

WHERE COUNT(*) > 2 :

- A) Est correct partout
- B) Est en general invalide (agregat)
- C) Remplace JOIN

Question 4

EXISTS teste :

- A) Une somme
- B) L'existence d'au moins une ligne
- C) Un type de colonne

Question 5

CASE WHEN sert a :

- A) Creer des branches / etiquettes

- B) Supprimer une table
- C) Remplacer FROM

Question 6

Pour un mois calendaire, une bonne idee est :

- A) Des bornes de dates claires
- B) Aucun filtre
- C) LIKE sur le montant

Question 7

UNION exige surtout :

- A) Des SELECT de forme compatible
- B) Des index
- C) Du HTML

Question 8

Une vue :

- A) Est une requete nommee
- B) Est un antivirus
- C) Remplace les sauvegardes

Question 9

NOT IN avec NULL dans la liste :

- A) Est toujours simple
- B) Peut surprendre / preferer NOT EXISTS
- C) Cree une table

Question 10

ROW_NUMBER (idee fenetre) :

- A) Ecrase toujours les lignes comme GROUP BY
- B) Peut numeroter en gardant le detail
- C) Supprime HAVING

Question 11

Bonne pratique sous-requete :

- A) Tout coller sans test
- B) Tester le SELECT interieur d'abord
- C) Eviter les alias

Question 12

Rapport mensuel :

- A) Sans verifier l'echantillon
- B) Avec periode + verification
- C) Uniquement SELECT *

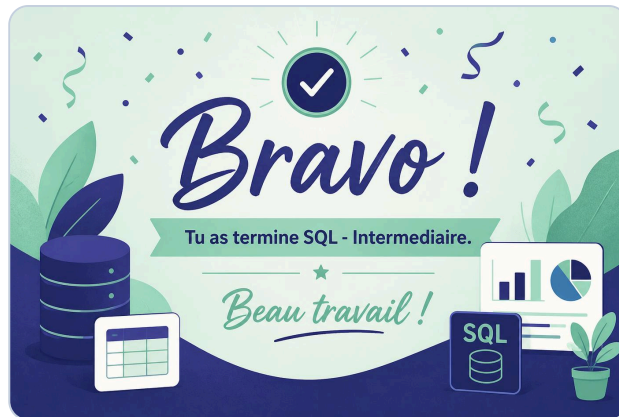
Corrige

1B 2B 3B 4B 5A 6A 7A 8A 9B 10B 11B 12B

★ A RETENIR

Sous-requetes, HAVING, dates, verification.

Bravo



Bravo. Tu construis des rapports utiles.

Tu as termine **SQL - Intermediaire**.

Tu poses des questions plus fines : sous-requetes, dates, HAVING, unions, et tu as vu vues / index / transactions / fenetres en mode idee. Chez DanielCraft, c'est le niveau Inter du pack Data/SQL.

Suite

SQL Expert (perf, plans, fenetres avancees) plus tard. Ou croiser avec Python pratique. Ou appliquer le mini-projet sur de vraies donnees anonymisees.

★ A RETENIR

Puissance + bornes + echantillon.

♦ ASTUCE

Relis HAVING et dates avant le prochain rapport.

Tu as les briques. Maintenant, interroge.